COMP1549 Coursework: Role-Based Access Control System Implementation and Report

Group 23 Members:

- Henriette Marina Banu: 001392967
- Gizem Kumrili: 001313271
- Mo Abdullah: 001431156
- Tonny Andino Munoz: 001393282

Table 1 - Java Generics implementation:

Generic Concept Used: Type-safe capability-based access control using Java Generics
Involved classes/methods • Capability <T> generic interface • ReadCapability implements Capability<Action> • WriteCapability implements Capability<Action> • AccessControlPolicyEngine.evaluateCapability(User, Resources, Capability<Action>)
Justification: The capability system utilizes the Capability<T> interface with the generic structure to enable the development of an effective and safe structure that can be used to define the various permission types. The generic concept was chosen enables each capability, such as ReadCapability and WriteCapability, to define its own behaviour without compromising the ability to meet the common abstraction. The capability system utilizes the CapabilityManager class to fix the type parameter to Action. The chosen concept is important in the development of a safe and effective capability system because it prevents the possibility of invalid capability usage. Therefore, we used this generic concept since it provides an effective and safe way of modelling the capability system without the utilization of Enum's and strings.

Table 2 - Design patterns implementation:

Pattern: Singleton **Involved classes/methods:** AccessControlPolicyEngine.getInstance() **Justification:** The Singleton pattern was chosen because it provides a dependency-free way to ensure that the access-control policy engine exists as one consistent instance across the whole system, which is important for applying policies and logging decisions in a uniform way. The Factory pattern, the Builder pattern, the Prototype pattern and the Object Pool pattern is more appropriate in scenarios where more than one object is to be created, which is not the case in the access control policy scenario because it is essential to have only one access control policy in the system. The Singleton pattern is the most suitable in ensuring that all operations related to accessing the access control policy are performed in one place and in a consistent way, without risks of having different versions of the access control policy in different parts of the system.
Pattern: Factory **Involved classes/methods:** UserFactory.java : createStudent(), createStaff(), createAdmin() **Justification:** This class is used to centralize the creation of the User class to ensure consistency in the instantiation of the class throughout the system. The class ensures that the IDs are correctly formatted and unique for all types of users. The Builder pattern is not suitable because it is unnecessarily complex when only two data members are present in the class. The Prototype pattern is not suitable because it might lead to duplicate IDs of the user. The static method in the User class is not used to maintain the single responsibility principle of OOP, which is not preferred in the given problem. The presence of resetCounters() ensures that the IDs can be reset during Junit testing in such a way that other tests are not affected. GRASP patterns like Creator or Information Expert were not used in this case as it would result in giving responsibilities to the User class for ID creation. Therefore, a factory pattern was the most suitable approach for this system.
Pattern: Strategy-like role-based decision logic **Involved classes/methods:** AccessControlPolicyEngine.evaluateAccess()

Justification: Depending on the roles of the user and the sensitivity of the resources, different strategies are used for access decisions, and this supports a modular and extensible approach for access control. The Observer pattern is geared toward notifications of state changes and access control involves rule evaluation, not notifications. The Visitor pattern is geared toward performing complex operations on complex object structures and access control decisions involve a simple structure of user, resource and action, not a complex structure. The Decorator pattern involves dynamic addition of responsibilities to objects and this does not match the need for selecting and applying the appropriate logic for access evaluation. Therefore, strategy pattern was chosen since it's role-based decision strategies and capabilities is clearer, more direct and more maintainable for access control.

Table 3 - Testing implementation:

Test: Admin permission validation **Involved classes/methods:** AccessControlPolicyEngine.evaluateAccess(), Role, Resources, Action **Description:** This test verifies that ADMIN user is granted or denied access correctly based on the defined RBAC. The test checks if the decision returned is according to the RBAC rules defined for the ADMIN role. The test compares the expected outcome with the actual outcome to ensure that the policy engine is correctly interpreting the ADMIN role's access. This ensures the correctness of the RBAC implementation for the most privileged role.
Test: Student restricted access enforcement **Involved classes/methods:** AccessControlPolicyEngine.evaluateAccess(), Role, Resources **Description:** This test ensures correct denial behaviour according to RBAC rules. This test tries to access certain resources (e.g., exam papers) that are protected and uses the role STUDENT. This test checks whether the result of calling the evaluateAccess() method always denies access. This confirms that the system enforces the RBAC rule that users of the role STUDENT are not allowed to access resources that are sensitive or are of modification level. This test, therefore, tests the system's ability to prevent privilege escalation
Test: Invalid input fail-safe behaviour (non-trivial) **Involved classes/methods:** AccessControlPolicyEngine.evaluateAccess() **Description:** The test function intentionally passes invalid parameters (null user, null resource, null action) to evaluateAccess(). It verifies that evaluateAccess() correctly returns denial instead of throwing an exception or granting access. It confirms robustness and fault tolerance of the policy engine.
Test: Access logging verification (non-trivial) **Involved classes/methods:** AccessControlPolicyEngine.evaluateAccess(), AccessLogEntry **Description:** The test invokes one or several access requests and checks the resulting AccessLogEntry data. The test checks that all access requests, regardless of the allow/deny result, are recorded with the appropriate data. This ensures that the logging system is always called by the evaluateAccess() method and that traceability and auditability requirements are satisfied.

Table 4 - Fault tolerance implementation:

Rule: Admin full access **Involved classes/methods:** AccessControlPolicyEngine.evaluateAccess(), Role Enum **Description: The** Role.ADMIN branch in the evaluateAccess() bypasses the checks and returns true for all combinations of resources and actions. This ensures that users with the ADMIN role are granted full read/write access to the system.
Rule: Staff Restricted access **Involved classes/methods:** AccessControlPolicyEngine.evaluateAccess(), Role Enum, Resource Enum. **Description:** The Role.STAFF logic verifies the requested resource and action against a predefined list of allowed operations (e.g., write lecture materials). Requests to modify/write for confidential scope resources (e.g. exam papers) are outside the allowed operations and are therefore denied. This policy ensures role-based separation of privileges while maintaining operational functionality.
Rule: Student limited access

Involved classes/methods: AccessControlPolicyEngine.evaluateAccess(), Role Enum **Description:** STUDENT users can only perform read operations on public and internal resources and are denied writing or write/read privileges for confidential resources. This ensures strict least-privilege behaviour.
Rule: Invalid request handling (non-trivial) **Involved classes/methods:** AccessControlPolicyEngine.evaluateAccess() **Description:** The evaluateAccess() method checks if the user, resource and action are not null. If the parameters are invalid, the access request is denied immediately. This is done to avoid undefined actions and to maintain system integrity.
Rule: Access logging enforcement **Involved classes/methods:** AccessControlPolicyEngine.getLogs() **Description:** The user, resource, action, and decision are logged every time the evaluateAccess() method is called to ensure traceability and auditing.
Rule: Concurrent Access Conflict Handling (non-trivial) **Involved classes/methods:** AccessTask, AccessResult **Description:** For this purpose, an ADMIN user, a STAFF member and a STUDENT attempt to access the same resource simultaneously. To avoid any conflicting states in the system, the access control policy engine implements a rule to prevent such access. When conflicting access attempts are identified, all such access attempts are denied. This approach is in line with the principles of designing a secure system, which avoids any priority-based decisions that could inadvertently lead to privilege escalation or compromise the integrity of the system. Instead, the system follows a consistent pattern of denying access in scenarios where conflicting access attempts are identified. This approach maintains the integrity of the system, which can be observed in the execution of the program with "Scenario 2: Same Resource Conflict."

AI Program/Tool: Chat GPT (Open AI) and CoPilot **Involved components:** Conceptual guidance related to AccessControlPolicyEngine design, testing strategy and debugging. **Contribution: ~20%** (Used exclusively to provide conceptual support, including explaining test logic and helping to understand design alternatives. AI was used as an extra tutor. Majority of code implementation and all of report writing and planning was done together as a group.)

References: (Most of our technical information is derived from sources such as GeeksforGeeks and W3Schools. However, only one link of each source is included in the reference section to maintain clarity and relevance due to page limit constraints.)

- Bishop, M. (2018) Computer Security: Art and Science. 2nd edn. Addison-Wesley.
- Gamma, E., Helm, R., Johnson, R. and Vlissides, J. (1994) Design Patterns: Elements of Reusable Object-Oriented Software. Addison-Wesley.
- Khan, J.A. (2024) Role-Based Access Control (RBAC) and Attribute-Based Access Control (ABAC). IGI Global.
- Oracle (n.d.) Enum Types. Available at: https://docs.oracle.com/javase/tutorial/java/javaOO/enum.html
- OpenAI (2025) ChatGPT. Available at: https://openai.com
- Eclipse Foundation (n.d.) Eclipse Platform Documentation. Available at: https://help.eclipse.org
- GeeksforGeeks (2019) Static Variables in Java. Available at: https://www.geeksforgeeks.org/java/static-variables-in-java/
- W3Schools (2020) Java Methods. Available at: https://www.w3schools.com/java/java_methods.asp